

基于本地大语言模型的法律法规 智能问答系统

技术报告

组长：徐博 2023040197

组员：张祖炫 2022060248

目录

1. 项目概述	3
2. 系统架构	4
3. 目录结构	5
4. 数据处理流程	7
5. 检索层 (rag/retriever.py)	9
6. 多 Agent 协作层 (agents/)	9
7. Web 服务与前端	11
8. 评估体系 (evaluation/)	12
9. 部署与运行	13
10. 关键配置项 (config.py)	14
11. 设计亮点与注意事项	14
12. 后续可扩展方向	15

版本: 1.0 **适用对象:** 开发人员 / 测试人员 / 交付评审

关键词: 本地 LLM、RAG、FAISS、LangChain、多 Agent、法律
法规问答

1. 项目概述

本项目构建了一个**完全本地化、离线运行**的法律法规智能问答系统。系统以《国家法律法规数据库》（flk.npc.gov.cn）为主要数据源，自爬法律法规文本，构建本地知识库，结合向量检索（FAISS）与本地大语言模型（Ollama + Qwen2.5），通过 LangChain 多 Agent 协作实现”检索—问答—总结”一体化服务。

1.1 设计目标

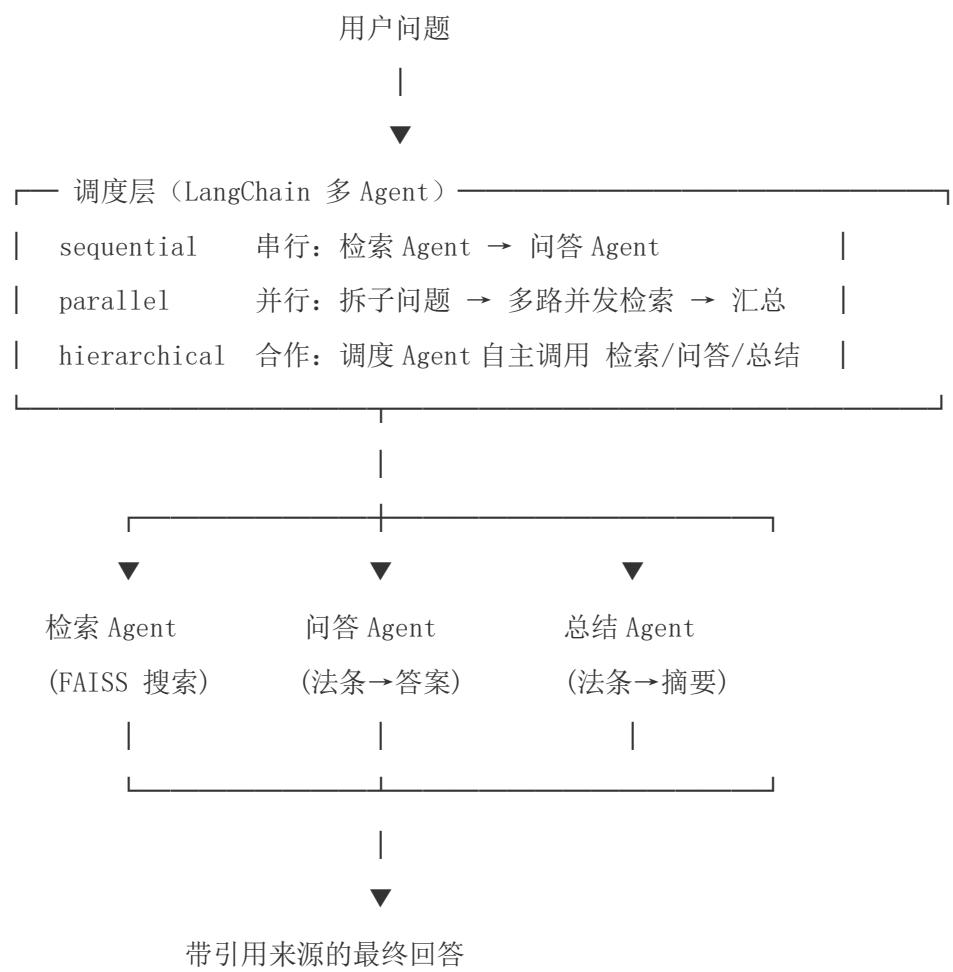
目标	说明
数据本地化	法律文本、向量索引、LLM 均在本地，全程不依赖外部云端 API
可溯源	回答必须附《法律名》第 X 条引用，禁止模型编造法条
多 Agent 协作	支持串行 / 并行 / 合作三种调度模式
多轮上下文	通过 session_id 维持对话历史，支持追问
易交付	提供 FastAPI 接口 + Streamlit 前端 + Swagger 文档

1.2 核心指标

1. 数据规模：84 部核心法律，约 **8798** 条分段，**8835** 条向量
2. 嵌入模型：BAAI/bge-large-zh-v1.5（1024 维，L2 归一化）
3. 检索索引：FAISS IndexFlatIP（归一化后内积等价于余弦相似度）
4. 生成模型：Ollama 部署 Qwen2.5（3b / 7b / 14b 可切换）
5. 测试集：102 条人工标注问题，覆盖 10 个法律类别

2. 系统架构

系统采用**四层架构**：数据层 → 检索层 → 生成层 → 调度层。



2.1 技术栈

层级	技术选型	关键库
数据爬取	Python HTTP + DOCX 解析	requests、python-docx、BeautifulSoup
文本处理	正则分段 / 分块	标准库 re
向量化	语义嵌入模型	sentence-transformers
向量检索	近似最近邻索引	faiss-cpu

层级	技术选型	关键库
生成	本地推理	ollama、langchain-ollama
Agent 框架	LangGraph Agent + LCEL	langchain / langchain-core
Web 服务	RESTful API	fastapi、uvicorn
前端	交互界面	streamlit
评估	批量跑批 + LLM 自评	argparse、ollama

3. 目录结构

```

.
├── config.py                # 全局配置（路径/爬虫/向量/模型）
├── requirements.txt         # 依赖清单
├── setup.bat               # 环境初始化脚本
├──
├── crawler/               # 数据爬取层
│   ├── npc_crawler.py     # 国家法律法规数据库爬虫（主数据源）
│   ├── pkulaw_crawler.py  # 北大法宝爬虫（Selenium，备选）
│   ├── cleaner.py         # 清洗 + 按法条分段 + 类型过滤
│   └── utils.py           # UA 池 / 延时 / 日志 / JSON 读写
├──
├── rag/                   # 检索增强生成层
│   ├── embeddings.py      # bge 模型封装（CPU 推理）
│   ├── indexer.py         # FAISS 索引构建 / 加载 / 保存
│   ├── chunker.py         # 法条分块（长拆项 / 短扩写）
│   ├── retriever.py       # 向量检索 + 同法条折叠去重
│   └── prompts.py         # Prompt 模板（Agent 与评估共用）
├──

```

```

├── agents/                                # 多 Agent 协作层
│   ├── tools.py                          # law_search 工具 + 检索器单例
│   └── orchestrator.py                   # LangChain 编排器（三模式）
│
├── api/
│   └── server.py                         # FastAPI 服务（预加载索引）
│
├── frontend/
│   └── streamlit_app.py                  # Streamlit 前端
│
├── evaluation/                            # 测试评估
│   ├── test_questions.json               # 102 条标注问题
│   ├── test_results.json                 # 评估结果（自动生成）
│   ├── run_eval.py                       # 批量评估脚本
│   └── auto_eval.py                      # LLM 自动评分
│
├── scripts/                              # 运维入口
│   ├── run_crawler.py                   # 爬虫入口
│   ├── build_index.py                   # 构建索引
│   ├── verify_index.py                  # 索引验证
│   ├── test_agents.py                   # Agent 测试
│   ├── test_api.py                      # API 测试
│   ├── run_api.py                       # API 启动
│   └── analyze_segments.py              # 分段分析
│
├── data/
│   ├── raw/npc/                          # 爬虫原始 JSON + TXT
│   ├── processed/                        # segments / chunks
│   └── vectors/                          # FAISS 索引 + 元数据
│
└── logs/                                # 运行日志

```

4. 数据处理流程

4.1 阶段一：数据爬取（crawler/npc_crawler.py）

主数据源为国家法律法规数据库官方二期 API（无需登录、免费）。流程分三步：

1. **搜索** POST /law-search/search/list —— 传入关键词/类型，返回法规
bbbs 唯一 ID
2. **详情** GET /law-search/search/flfgDetails —— 获取元数据 + 文件路径
3. **下载** GET /law-search/download/mobile?format=docx —— 移动端接口直返
DOCX 二进制流，用 python-docx 解析正文

关键设计： - **智能限流**：每请求 0.5s 基础延时，每 15 次触发 3s 长延时避免被封 - **重试机制**：指数退避重试（ $2/2^2/2^3$ s） - **客户端类型过滤**：按法律类型名（flxz）保留“宪法 + 法律”，过滤地方法规/司法解释/案例 - **时效性过滤**：默认只搜 sxx=3（现行有效） - **去重**：按 title 去重后保存为 JSON + 单条 TXT

备选数据源 pkulaw_crawler.py 使用 Selenium 模拟浏览器爬取北大法宝，需 Chrome + WebDriver，适合补充未覆盖内容。

对应入口：scripts/run_crawler.py

```
python scripts/run_crawler.py --source npc --keywords 民法典 劳动法 --clean
python scripts/run_crawler.py --source npc --type law --max 50 --clean
```

4.2 阶段二：清洗与分段（crawler/cleaner.py）

DataCleaner 对原始 JSON 逐条处理： - **清洗**：去除特殊不可见字符（零宽字符、BOM）、统一换行、去除 HTML 残留标签、压缩多余空格 - **按法条切分**
split_by_article：以“第 X 条”正则定位切分点，保留“第 X 条 + 正文”格式；无法条标记的文本按段落切分 - **类型过滤**：白名单[“宪法”，“法律”] 只保留核心法典 - **去重**：按 title + article_no 去重 - **输出**：data/processed/npc_segments.json

4.3 阶段三：智能分块（rag/chunker.py）

LegalChunker 在分段基础上做语义分块，遵循两条原则：

原则一：长法条切块但不丢魂 (>500 字) - 优先按”项” ((一) (二) ...) 拆分 - 单”项” 仍超长则按句/段 (。；) 二次拆分 - 每个子块强制保留 《法律名》第 X 条 前缀 - 记录元数据: chunk_index、chunk_total、parent_article_id

原则二：短法条扩写但不篡改 (<150 字) - content 原样保留 (展示纯净) - 额外生成 embedding_text = ”《法律名》第 X 条: 正文” 仅用于向量编码, 提升短条款召回率

输出 data/processed/npc_chunks.json, 每个 chunk 含 content (展示用)、embedding_text (编码用)、chunk_type (split/short/whole) 等字段。

4.4 阶段四：向量化与索引 (rag/embeddings.py + rag/indexer.py)

- **嵌入模型:** EmbeddingModel 封装 sentence-transformers, 运行在 CPU (省显存给 LLM), normalize_embeddings=True
- **编码:** 优先使用 chunk 的 embedding_text, 否则 fallback 到 content
- **索引构建:** FaissIndexer.build() 使用 IndexFlatIP——向量归一化后内积等价于余弦相似度
- **持久化:** {name}.faiss (向量) + {name}_metadata.json (chunk 元数据, 检索结果回填用)
- 默认 top_k=5, 加载时通过 get_retriever() 单例复用, 避免重复加载 embedding 模型

对应入口: scripts/build_index.py

```
python scripts/build_index.py --source npc
```

```
python scripts/verify_index.py          # 验证索引可用性
```

5. 检索层 (rag/retriever.py)

FaissRetriever 是检索核心，采用同法条折叠去重策略：

1. 从 FAISS 检索 $2 \times \text{top_k}$ 条候选（多取以补偿折叠损失）
2. 按 parent_article_id 折叠：同一条法的多个 chunk 合并内容（拼接），分数取最高分
3. 去重排序，截取 top_k 返回 (chunks, scores)

折叠效果：避免同一长法条的多个子块在上下文中重复出现，节省 token 并提升信息密度。

```
retriever = FaissRetriever()
chunks, scores = retriever.search("试用期最长可以约定多久?", top_k=5)
```

6. 多 Agent 协作层 (agents/)

6.1 工具定义 (agents/tools.py)

- law_search(query): LangChain @tool，内部调用共享检索器单例，复用 build_context_text 统一格式返回法条
- get_retriever(): 模块级单例，供 API / Agent / 评估脚本共用，避免重复加载模型

6.2 编排器 (agents/orchestrator.py)

AgentOrchestrator 基于 LangChain 1.x (create_agent 返回 LangGraph agent)，提供三种调度模式：

模式	方法	流程
sequential	_run_sequential	检索 Agent (ReAct) 拿法条 → 问答 Agent (LCEL chain) 生成答案

模式	方法	流程
parallel	<code>_run_parallel</code>	LLM 拆 2~3 子问题 → ThreadPoolExecutor 真并行检索 → 问答 Agent 汇总
hierarchical	<code>_run_hierarchical</code>	调度 Agent 把 检索/问答/总结 包装为 3 个工具，LLM 自主决策调用

Agent 构成： - **检索 Agent**: `create_agent + law_search` 工具，ReAct 范式，自主决定检索关键词 - **问答 Agent**: `LCEL chain QA_TASK_TEMPLATE | llm | StrOutputParser` - **总结 Agent**: `LCEL chain SUMMARY_TASK_TEMPLATE | llm | StrOutputParser` - **调度 Agent**（合作模式）：把上述三个封装为工具，由 LLM 决定调用顺序

6.3 Prompt 统一管理（rag/prompts.py）

所有 Prompt 集中管理，Agent 与评估脚本共用，保证法条格式一致： - `REWRITE_PROMPT`: 并行模式查询重写 - `build_context_text()`: 将 chunk 列表拼为 [i] 《XX 法》第 X 条\n 内容 格式 - `RETRIEVAL_TASK_TEMPLATE / QA_TASK_TEMPLATE / SUMMARY_TASK_TEMPLATE`: 三类任务指令

`QA_TASK_TEMPLATE` 内置 6 条硬性规则：必须依据检索法条、标注引用、无依据时说明“暂无法确定”、简洁、加专业提醒、优先相关领域条文。

6.4 多轮对话

`chat()` 方法通过 `session_id` 维护对话历史： - 每次对话保留最近 10 轮（传入时取近 5 轮拼为 `history_text`） - 历史作为上下文前缀注入，支持基于上文的追问 - 前端为每浏览器会话生成唯一 `session_id`，后端按 `session_id` 隔离

7. Web 服务与前端

7.1 FastAPI（api/server.py）

启动即预加载索引与编排器（`get_retriever` / `get_orchestrator` 懒加载单例），后续请求秒级响应。

端点	方法	说明
<code>/chat</code>	POST	多 Agent 问答，参数： <code>question</code> 、 <code>mode</code> 、 <code>model</code> 、 <code>session_id</code>
<code>/search</code>	POST	纯检索（不调 LLM），返回 <code>top_k</code> 法条 + 相似度分数
<code>/health</code>	GET	健康检查，返回索引规模与维度
<code>/clear_session</code>	POST	清空指定会话历史

- 自动生成 Swagger UI：`http://localhost:8000/docs`
- 开启 CORS 中间件（开发期 `allow_origins=["*"]`）

启动：`python scripts/run_api.py`（默认 `0.0.0.0:8000`）

7.2 Streamlit 前端（frontend/streamlit_app.py）

- 侧边栏可选 **Qwen2.5 模型**（3b/7b/14b）与 **Agent 模式**（串行/并行/合作）
- “系统状态”按钮调用 `/health` 显示索引规模
- 提供 5 个示例问题一键填充
- 每会话生成唯一 `session_id`；“清空对话”同步调用 `/clear_session`
- 调用 `/chat` 展示多轮对话（markdown 渲染）

启动：`streamlit run frontend/streamlit_app.py`（默认 `http://localhost:8501`）

8. 评估体系 (evaluation/)

8.1 测试数据集

test_questions.json 含 102 条问题，字段符合需求模板：

```
{  
  "question": "试用期最长可以约定多久？",  
  "expected_answer": "根据《劳动合同法》第十九条，劳动合同试用期最长不得超过六个月。",  
  "category": "劳动法"  
}
```

覆盖劳动法、民法、刑法等 10 个法律类别。

8.2 批量评估 (run_eval.py)

复用与前端完全相同的链路 (AgentOrchestrator.ask + FaissRetriever)，逐条自动填充：

- retrieved_text: 检索到的法条
- model_output: Agent 最终回答
- evaluation_note: 留待自动评分

支持三种模式分别或 --mode all 对比运行，带断点续跑（按 question+mode 去重合并）。

```
python evaluation/run_eval.py --mode all --model qwen2.5:7b  
python evaluation/run_eval.py --mode parallel --start 0 --end 10
```

8.3 自动评分 (auto_eval.py)

用 LLM 对比 expected_answer 与 model_output，按“核心结论正确性 / 法条引用 / 明显错误 / 冗余含糊”生成 20 字内 evaluation_note，并写回 test_results.json。

```
python evaluation/auto_eval.py --model qwen2.5:14b
```

9. 部署与运行

9.1 环境要求

- Python 3.13
- Ollama 桌面版（本机运行 LLM）
- 可选 NVIDIA GPU（embedding 已在 CPU，LLM 可走 GPU）

9.2 安装

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
ollama pull qwen2.5:3b
ollama pull qwen2.5:7b    # 推荐
ollama pull qwen2.5:14b
```

国内访问 HuggingFace 较慢，config.py 默认设置 HF_ENDPOINT=https://hf-mirror.com 镜像；模型已缓存后开启 USE_HF_OFFLINE=True 避免联网校验卡顿。

9.3 一键启动

```
# 终端 1: API
python scripts/run_api.py
```

```
# 终端 2: 前端
streamlit run frontend/streamlit_app.py
```

访问 <http://localhost:8501> 使用前端，<http://localhost:8000/docs> 测试 API。

9.4 更新法条

```
python scripts/run_crawler.py --source npc --type law --keywords 新法名
--max 10 --clean
python scripts/build_index.py
```

10. 关键配置项（config.py）

配置	默认值	说明
USE_HF_OFFLINE	True	离线模式（模型已缓存后开启）
CRAWLER_CONFIG	delay 1~3s / max_retries 3	爬取节流与重试
NPC_CONFIG	flk.npc.gov.cn	主数据源 API 地址
SEARCH_KEYWORDS	20 个核心法典	默认爬虫关键词
EMBEDDING_CONFIG	bge-large-zh-v1.5 / cpu	嵌入模型
VECTOR_CONFIG	IndexFlatIP / top_k=5	检索索引类型与返回数

11. 设计亮点与注意事项

亮点 - **离线隐私**：全链路本地，数据不出域 - **长/短法条差异化处理**：长法条拆项保前缀、短法条扩写下文，兼顾召回与精度 - **同法条折叠**：避免上下文重复浪费 - **统一 Prompt 源**：Agent 与评估共用 rag/prompts.py，格式一致可追溯 - **真并行检索**：ThreadPoolExecutor 并发多路检索，非伪并行

注意事项 - npc_crawler.py 需关闭 SSL 校验（verify=False），生产环境应注意合规风险 - 北大法宝爬虫需登录/付费内容，且选择器可能随网站改版失效 - server.py 开发期 CORS 全开，生产应限制 allow_origins - 并行模式依赖 LLM 拆问题的能力，拆分失败会自动降级为串行 - embedding 固定 CPU，首条检索因加载模型会有冷启动延迟（单例后消除）

12. 后续可扩展方向

1. 引入重排序（rerank）模型提升检索精度
2. 支持更多数据源（裁判文书网、司法解释）
3. 增加检索结果可信度评分与引用跳转
4. 评估指标量化（准确率 / 引用命中率 / 自动评分一致性）
5. 容器化部署（Docker）与并发优化